# Tests unitaires de China Map Quiz

Version 2 (en cours) — 151 tests dans 14 fichiers.

Ce document décrit chaque test unitaire de l'application : ce qu'il vérifie et pourquoi. Comme tout le projet, les tests et ce document ont été écrits par Claude Code (Anthropic), une IA générative, à la demande de l'auteur du projet.

## Comment ces tests fonctionnent

**Où ils sont.** Dans `app/src/test/java/io/github/saksa84/chinamapquiz/`, rangés comme le code qu'ils vérifient : `data`, `game`, `map`, `progress`.

**Comment les lancer.** Depuis le dossier du projet, en PowerShell :

```
$env:JAVA_HOME = "C:\Program Files\Android\Android Studio\jbr"
.\gradlew.bat testDebugUnitTest
```

Le rapport détaillé s'ouvre ensuite dans un navigateur : `app/build/reports/tests/testDebugUnitTest/index.html`.

**Pourquoi ils tournent sans téléphone.** Ce sont des tests JUnit exécutés par la machine Java du PC, en quelques secondes. C'est possible parce que les règles du jeu (`game`), la géométrie de la carte (`map`), la progression (`progress`) et la validation des données (`data`) sont écrites en Kotlin pur, sans dépendre d'Android. Les écrans, la synthèse vocale et le stockage sur le téléphone ne sont pas couverts par ces tests : ils sont vérifiés à la main, sur l'émulateur et sur le téléphone.

**Deux familles de tests.**

- Les tests **sur des exemples fabriqués** vérifient une règle précise avec des données minimales écrites dans le test (par exemple : un affluent rattaché à une ville doit être refusé).
- Les tests **sur les vraies données** (fichiers dont le nom commence par `Real`, et quelques autres) lisent les fichiers réellement livrés dans l'application (`app/src/main/assets`). Ils servent de garde-fous : une erreur de saisie dans les données fait échouer les tests.

**Convention de nommage.** Le nom d'un test décrit le comportement attendu, en anglais : `duplicateId_isRejected` se lit « un identifiant en double est refusé ».

## Récapitulatif

| Fichier | Tests | Domaine |
|---|---|---|
| `data/GeoCatalogLoaderTest` | 45 | Lecture et validation des fichiers de lieux |
| `data/RealAssetsTest` | 9 | Vraies données : unités provinciales, régions, îles |
| `data/RealFeaturesTest` | 16 | Vraies données : villes, cours d'eau, montagnes, lacs, parcs |
| `data/PlaceFactsTest` | 5 | Fiches « En savoir plus » |
| `game/AnswerCheckerTest` | 10 | Validation des réponses tapées (mode Saisir) |
| `game/QuizSessionTest` | 14 | Moteur des modes Trouver et Saisir |
| `game/EliminateSessionTest` | 7 | Moteur du mode Éliminer |
| `game/StopwatchTest` | 3 | Chronomètre avec pause |
| `game/QuizLevelsTest` | 6 | Niveaux de jeu |

| Fichier | Tests | Domaine |
|---|---|---|
| `map/HitTesterTest` | 14 | Détection du toucher sur la carte |
| `map/MapViewportTest` | 5 | Zoom et déplacement |
| `map/RealMapTest` | 7 | Vraie carte et vraies coordonnées GPS |
| `progress/ProgressTest` | 9 | Records, erreurs fréquentes, sauvegarde |
| `ExampleUnitTest` | 1 | Exemple du modèle Android Studio |
| **Total** | **151** | |

## data/GeoCatalogLoaderTest — lecture et validation des données (45 tests)

Le chargeur (`GeoCatalogLoader`) lit les fichiers JSON des lieux et les valide ensemble avant que l'application ne s'en serve. Chaque règle de validation a au moins un test qui la fait échouer volontairement, pour vérifier que l'erreur est bien détectée et bien nommée. Les premiers tests utilisent de petits fichiers d'exemple (« fixtures ») placés dans `app/src/test/resources/geo`.

| Test | Ce qu'il vérifie |
|---|---|
| `fixtures_loadAllCategories` | Les fichiers d'exemple se chargent sans erreur et contiennent chaque catégorie de lieu (pays, provinces, régions, villes, fleuves, montagnes). |
| `fixtures_provincesAreChildrenOfChina` | Les provinces d'exemple sont bien rattachées à la Chine. |
| `fixtures_keepTypesAliasesAndMetadata` | Le type, les alias et les métadonnées (hanzi, capitale…) sont conservés tels quels après chargement. |
| `fixtures_geometryKindDependsOnType` | La forme attendue dépend du type : surface pour une province, ligne pour un fleuve, point pour une ville ou un sommet. |
| `fixtures_regionsResolveTheirProvinces` | Une grande région retrouve la liste de ses provinces membres. |
| `fixtures_riversAndMountainsHaveTheirOwnHierarchy` | Les affluents sont rattachés à leur fleuve et les sommets à leur chaîne, et non à une province. |
| `lookup_unknownIdReturnsEmptyResults` | Chercher un identifiant inconnu renvoie un résultat vide au lieu de planter. |
| `citiesFileAlone_failsBecauseItsProvinceIsMissing` | Le fichier des villes seul est refusé : ses villes pointent vers des provinces absentes. Les fichiers doivent être validés ensemble. |
| `sameIdInTwoFiles_isRejected` | Un même identifiant présent dans deux fichiers différents est refusé. |
| `conflictingSourcesAcrossFiles_areRejected` | Une source de données déclarée dans deux fichiers avec des informations différentes est refusée. |
| `malformedJson_isReported` | Un JSON illisible produit un message d'erreur, pas un plantage. |
| `unknownFeatureType_isReported` | Un type de lieu inconnu est signalé. |

| Test | Ce qu'il vérifie |
|---|---|
| `missingRequiredField_isReported` | Un champ obligatoire manquant (identifiant, nom…) est signalé. |
| `validMinimalFile_hasNoErrors` | Le plus petit fichier valide possible passe sans erreur : les règles ne sont pas trop strictes. |
| `duplicateId_isRejected` | Deux lieux avec le même identifiant sont refusés. |
| `unknownSource_isRejected` | Un lieu qui cite une source non déclarée est refusé : chaque donnée doit avoir une provenance. |
| `unsupportedSchemaVersion_isRejected` | Un fichier d'une version de format inconnue est refusé. |
| `unknownParent_isRejected` | Un lieu rattaché à un parent inexistant est refusé. |
| `missingRequiredParent_isRejected` | Un lieu qui doit avoir un parent (une province, une ville) mais n'en a pas est refusé. |
| `provinceUnderProvince_isRejected` | Une province rattachée à une autre province est refusée. |
| `countryWithParent_isRejected` | Un pays ne peut pas avoir de parent. |
| `tributaryOfACity_isRejected` | Un affluent doit se jeter dans un fleuve, pas dans une ville. |
| `cityDirectlyUnderCountry_isRejected` | Une ville doit appartenir à une unité provinciale, pas directement au pays. |
| `peakWithoutRange_isAccepted` | Un sommet isolé, sans chaîne, est accepté (Tai Shan, Huang Shan…). |
| `selfParent_isRejected` | Un lieu ne peut pas être son propre parent. |
| `regionWithoutMembers_isRejected` | Une grande région sans province est refusée. |
| `regionWithUnknownOrNonProvincialMember_isRejected` | Une grande région ne peut contenir que des unités provinciales existantes. |
| `membersOnNonRegion_areRejected` | Seules les grandes régions peuvent avoir des membres. |
| `subRegionMemberOutsideParentRegion_isRejected` | Une sous-région (Huazhong) ne peut contenir que des provinces de sa région parente (Zhongnan). |
| `provinceInNoRegionOrInTwoRegions_isRejected` | Chaque province appartient à exactement une grande région : ni zéro, ni deux. |
| `chineseCharactersAsPinyin_areRejected` | Un nom affiché en caractères chinois est refusé : les noms des jeux doivent être en alphabet latin. |
| `toneMarks_areRejected` | Un nom principal avec des tons (Sìchuān) est refusé : les tons vont dans un champ à part. |
| `blankName_isRejected` | Un nom vide est refusé. |
| `apostrophesSpacesAndHyphens_areAccepted` | Apostrophes, espaces et traits d'union sont acceptés (Xi'an, Chang Jiang, Altyn-Tagh). |

| Test | Ce qu'il vérifie |
|---|---|
| `disputedIslandsWithIsoClaimants_areAccepted` | Un territoire disputé avec des pays revendicateurs en codes ISO (CN, TW, VN) est accepté. |
| `claimantThatIsNotAnIsoCode_isRejected` | Un pays revendicateur écrit en toutes lettres (« Vietnam ») au lieu d'un code ISO est refusé. |
| `islandGroupAttachedToACountry_isRejected` | Un territoire disputé ne doit pas être rattaché à un pays : c'est la règle de neutralité du projet. |
| `riverLengths_areChecked` | La longueur d'un fleuve doit être positive, et sa partie chinoise ne peut pas dépasser la longueur totale. |
| `knownAs_allowsAccents_butNameDoesNot` | Le nom connu accepte les accents (« Mékong ») et s'affiche « Mékong / Lancang Jiang » ; le nom pinyin, lui, les refuse. |
| `knownAsThatRepeatsTheNameOrUsesHanzi_isRejected` | Un nom connu qui répète le pinyin aux accents près (Taïwan / Taiwan), en hanzi, ou vide, est refusé. |
| `pinyinTonesInHanzi_isRejected` | Le champ « pinyin avec tons » ne peut pas contenir de caractères chinois. |
| `lengthOnSomethingOtherThanARiver_isRejected` | Une longueur n'a de sens que pour un cours d'eau. |
| `lakeArea_isCheckedAndReservedToLakes` | La superficie d'un lac est lue ; une superficie nulle, ou donnée à autre chose qu'un lac, est refusée. |
| `parkDesignations_areRequiredForParksOnly` | Les statuts d'un parc (« patrimoine mondial, 1992 ») sont lus ; un parc sans statut, ou un lac avec un statut, est refusé. |
| `lakeOutsideAProvince_isRejected` | Un lac doit être rattaché à une unité provinciale. |

## data/RealAssetsTest — vraies unités provinciales, régions et îles (9 tests)

Ces tests lisent les vrais fichiers `provinces.json`, `regions.json`, `islands.json` et `map.geometry.json`, et vérifient les décisions de périmètre prises pour le projet.

| Test | Ce qu'il vérifie |
|---|---|
| `has34ProvincialUnitsWithExpectedTypes` | Il y a exactement 34 unités provinciales : 23 provinces (22 et Taïwan, comme dans la norme ISO 3166-2:CN), 5 régions autonomes, 4 municipalités et 2 régions administratives spéciales. |
| `provincialIdsFollowIso31662` | Les identifiants suivent le format ISO 3166-2 (`CN-SC` pour le Sichuan). |
| `provincesAndAutonomousRegionsHaveACapital` | Chaque province et région autonome a une capitale renseignée. |
| `everyEntityHasHanzi` | Chaque lieu a son nom en caractères chinois, indispensable au bouton de prononciation. |

| Test | Ce qu'il vérifie |
|---|---|
| `regionsMatchTheDecidedLayout` | Les 6 grandes régions et la subdivision de Zhongnan (Huazhong, Huanan) correspondent au découpage décidé ; Taïwan est dans Huadong, Hong Kong et Macao dans Zhongnan, la Mongolie-Intérieure dans Huabei. |
| `hongKongAndMacaoUseTheirFrenchNameWithPinyinAsAlias` | Hong Kong et Macao s'affichent sous leur nom français, avec le pinyin (Xianggang, Aomen) comme alias. |
| `paracelIslandsAreDisputedAndNotAttachedToAnyCountry` | Les îles Paracel sont revendiquées par la Chine, Taïwan et le Vietnam, ne sont rattachées à aucun pays et ne comptent pas parmi les 34 unités. |
| `onlyDisputedTerritoriesHaveClaimants` | Seules les îles Paracel portent une mention « revendiqué par ». |
| `geometryCoversExactlyTheMapEntities` | Le fichier de carte contient exactement un tracé non vide par unité provinciale et pour les îles, sans doublon ni oubli. |

## data/RealFeaturesTest — vraies villes, cours d'eau, montagnes, lacs et parcs (16 tests)

Ces tests chargent toutes les données livrées, comme le fait l'application au démarrage, avec les tracés de `features.geometry.json` et la configuration des niveaux.

| Test | Ce qu'il vérifie |
|---|---|
| `everyFeatureHasGeometryOfTheRightKind` | Chaque ville est un point, chaque cours d'eau une ligne, chaque chaîne une surface, chaque sommet un point ; chaque lac une surface ; tous sont dessinés par-dessus les provinces. |
| `atLeast30CitiesWithPopulationAndAProvince` | Au moins 30 villes (exigence du cahier des charges), toutes de plus de 3 millions d'habitants et rattachées à une unité provinciale. |
| `citiesFallInTheirProvinceOnTheMap` | Le point de chaque ville tombe bien dans sa province sur la carte, avec 5 km de tolérance pour les villes côtières. |
| `tributariesFlowIntoAnotherRiver` | Chaque affluent se jette dans un cours d'eau ; le Dadu He se jette dans le Min Jiang. |
| `riversHaveALength_andTheYangtzeIsTheLongest` | Chaque cours d'eau a une longueur ; le Yangtsé est le plus long ; un affluent est plus court que son fleuve (sauf le Hai He, compté pour son seul cours principal) ; les fleuves transfrontaliers ont leur longueur en Chine. |

| Test | Ce qu'il vérifie |
|---|---|
| `riverSectionsLeaveNoGap` | Les tronçons d'un même cours d'eau se touchent (écart de moins de 30 km) : ce test aurait détecté le tronçon manquant du Zhu Jiang, corrigé avant la version 1.1. |
| `pinyinTonesMatchTheName` | Le pinyin avec tons, une fois les accents retirés, redonne le nom ou un alias, ce qui repère une faute de frappe dans les tons. Exceptions documentées : Shaanxi (Shǎnxī) et Yarlung Zangbo. |
| `peaksHaveAnElevation_andEverestIsTheHighest` | Chaque sommet a une altitude, et l'Everest est le plus haut. |
| `levelsOfEachCategoryHaveTheExpectedSizes` | Les niveaux des villes (15, 38, 40), des fleuves (12, 15, 27), des montagnes (17, 20, 37), des lacs (9, 15, 24) et des parcs (5, 11, 16) ont la taille prévue. |
| `theFiveNationalParks_areExactlyThoseOf2021` | Seuls Sanjiangyuan, Panda géant, Tigre et léopard, Forêt tropicale de Hainan et Wuyishan ont le statut de parc national, avec l'année 2021 : aucun site célèbre n'est présenté à tort comme parc national. |
| `parksHaveAContourFromOsm_exceptTheTwoMissingOnes` | Chaque parc a un contour, sauf les deux absents d'OpenStreetMap (des points), et il est dessiné dans sa province. |
| `lakesHaveAnAreaAndAProvince_andQinghaiIsTheLargestInChina` | Au moins 20 lacs, chacun avec sa superficie et sa province ; le lac Qinghai est le plus grand lac entièrement chinois (le lac Khanka, plus grand, est partagé avec la Russie) ; le plus haut dépasse 4 500 m. |
| `lakesAreDrawnInTheirProvince` | Chaque lac est dessiné dans sa province (15 km de tolérance), sauf le réservoir de Danjiangkou, à cheval sur le Hubei et le Henan. |
| `levelsOnlyContainPlacesOfTheirCategory` | Un niveau « Fleuves » ne pose que des questions sur des fleuves, et ainsi de suite. |
| `realCoordinatesHitTheRightFeature` | Un toucher aux vraies coordonnées de Wuhan trouve le Yangtsé, à Lanzhou le fleuve Jaune ; sur l'Everest, le sommet l'emporte sur la chaîne ; au milieu du lac Qinghai, de Xi Hu ou du côté chinois du lac Khanka, le bon lac ; à Jiuzhaigou, le site plutôt que le parc du Panda géant qui l'entoure ; en pleine mer, aucun cours d'eau. |

| Test | Ce qu'il vérifie |
|---|---|
| `typedNamesAreAcceptedForFeatures` | Le mode Saisir accepte « Yangtze », « fleuve jaune », « Everest », « Canton », « Mékong » ou « mekong », « Nam Co » et « lac des mille iles » ; l'affichage donne « Everest / Zhumulangma Feng » et « Namtso / Namu Cuo ». |

## data/PlaceFactsTest — fiches « En savoir plus » (5 tests)

Les fiches sont dans un fichier séparé, `facts.json`, rattaché aux lieux par leur identifiant.

| Test | Ce qu'il vérifie |
|---|---|
| `validFacts_areIndexedById` | Une fiche valide est retrouvée par l'identifiant de son lieu ; elle est marquée « à vérifier » tant qu'elle n'a pas été relue. |
| `unknownOrDuplicateIds_areRejected` | Une fiche pour un lieu inexistant, ou deux fiches pour le même lieu, sont refusées. |
| `emptyOrTooLongFacts_areRejected` | Une fiche vide, un texte de plus de 280 caractères ou plus de deux anecdotes sont refusés, pour que les fiches restent courtes. |
| `impossibleNumbers_areRejected` | Une population négative, une population sans année (ou l'inverse) et une superficie nulle sont refusées. |
| `realFacts_coverEveryPlace` | Le vrai fichier contient une fiche complète pour chacun des 138 lieux : population, superficie, langues, histoire, économie et plat pour les provinces ; rôle pour les villes ; présentation et anecdote pour les cours d'eau et les montagnes. Contrôle de vraisemblance : le Guangdong est la province la plus peuplée et le Xinjiang la plus étendue. |

## game/AnswerCheckerTest — réponses tapées, mode Saisir (10 tests)

`AnswerChecker` compare ce que le joueur tape au nom attendu, après normalisation. La règle du projet : on ignore la casse, les espaces, les tons, les accents, les apostrophes et les tirets, mais aucune autre faute n'est tolérée.

| Test | Ce qu'il vérifie |
|---|---|
| `normalize_ignoresCaseSpacesAndPunctuation` | «  SiChuan  » devient « sichuan », « Xi'an » devient « xian », « Hong-Kong » devient « hongkong ». |
| `normalize_removesTonesAndAccents` | « Sìchuān » devient « sichuan », « Lǔ » devient « lu », « Îles Paracel » devient « ilesparacel ». |
| `normalize_ofOnlyPunctuation_isEmpty` | Une saisie faite uniquement de ponctuation donne une chaîne vide, jamais acceptée. |
| `mainNameIsAccepted_inAnyCaseOrWithTones` | Le nom principal est accepté en minuscules, en majuscules ou avec les tons. |
| `declaredAliasesAreAccepted` | Les alias listés dans les données sont acceptés : « Tibet » pour Xizang, « Inner Mongolia », « Xianggang », « Macau », « Pékin ». |

| Test | Ce qu'il vérifie |
|---|---|
| `typosAreNotAccepted` | Une vraie faute d'orthographe (« Sichaun », « Sichua ») est refusée. |
| `shanxiAndShaanxiAreNeverConfused` | « Shanxi » et « Shaanxi », deux provinces voisines, ne sont jamais acceptées l'une pour l'autre. |
| `anotherPlaceNameIsWrong` | Le nom d'un autre lieu est une erreur. |
| `emptyOrUnknownIsWrong` | Une saisie vide, ou une question sur un lieu inconnu, donne « faux » sans planter. |
| `capitalsAreNotAcceptedAsProvinceNames` | La capitale (« Chengdu ») n'est pas acceptée à la place de la province (Sichuan). |

## game/QuizSessionTest — moteur des modes Trouver et Saisir (14 tests)

`QuizSession` déroule une partie : une question par lieu, une seule réponse par question, un chronomètre, puis un bilan. Le temps est simulé dans les tests, ce qui les rend instantanés et reproductibles.

| Test | Ce qu'il vérifie |
|---|---|
| `correctAnswer_scoresAndShowsFeedback` | Une bonne réponse rapporte un point, et la partie passe à la phase de retour (correction affichée). |
| `wrongAnswer_recordsWhatWasTouched` | Une erreur enregistre le lieu touché, pour la liste des erreurs. |
| `onlyOneAnswerPerQuestion` | Une seconde réponse à la même question est ignorée : une seule chance. |
| `next_goesThroughAllQuestionsThenFinishes` | « Suivant » passe par toutes les questions, puis la partie se termine. |
| `next_isIgnoredBeforeAnswering` | On ne peut pas passer une question sans y avoir répondu. |
| `emptySession_isImmediatelyFinished` | Une partie sans question est terminée d'emblée, sans planter. |
| `timer_runsDuringTheGameAndStopsAtTheLastAnswer` | Le chronomètre tourne pendant la partie et s'arrête à la dernière réponse. |
| `summary_listsMistakesInOrder` | Le bilan liste les erreurs dans l'ordre (lieu cherché, lieu touché). |
| `replayingMistakes_asksOnlyTheMissedPlaces` | « Rejouer les erreurs » ne pose que les questions manquées. |
| `shuffled_isReproducibleWithASeedAndAsksEachPlaceOnce` | Le tirage aléatoire pose chaque lieu une fois exactement, et le même germe redonne le même ordre. |
| `typedAnswers_keepTheTextAndTheCheckerVerdict` | En mode Saisir, le texte tapé et le verdict du vérificateur sont conservés. |
| `skippedQuestion_isAMistake` | Une question passée (bouton « Passer ») compte comme une erreur. |

| Test | Ce qu'il vérifie |
|---|---|
| `readingTheFacts_pausesTheTimerUntilTheNextQuestion` | Pendant la lecture de la fiche « En savoir plus », le chrono est en pause ; il repart à la question suivante, et le temps de lecture n'est pas compté. |
| `duplicateTargets_areRejected` | Une partie qui poserait deux fois le même lieu est refusée à la création. |

## game/EliminateSessionTest — moteur du mode Éliminer (7 tests)

`EliminateSession` applique les règles du mode Éliminer (version 1.2) : une seule chance par lieu, un lieu trouvé disparaît du jeu, un lieu manqué reste sur la carte.

| Test | Ce qu'il vérifie |
|---|---|
| `wrongTap_isAMistakeAndMovesOn_withoutEliminating` | Une erreur compte, passe à la consigne suivante, et le lieu manqué reste sur la carte. |
| `correctTap_eliminatesAndMovesOn` | Un bon toucher élimine le lieu et passe à la consigne suivante. |
| `eliminatedRegionsCannotBeTappedAgain` | Un lieu éliminé n'est plus touchable : le toucher n'est pas une réponse. |
| `gameEndsWhenEveryPlaceWasAskedOnce_andTimerStops` | La partie se termine quand chaque lieu a été demandé une fois, et le chronomètre s'arrête. |
| `onlyMissedPlacesRemainOnTheMapAtTheEnd` | À la fin, seuls les lieux manqués restent sur la carte. |
| `summary_countsSuccessesAndListsEachMistake` | Le bilan compte les lieux trouvés et liste chaque erreur (lieu cherché, lieu touché). |
| `emptyGame_isFinished` | Une partie sans lieu est terminée d'emblée. |

## game/StopwatchTest — chronomètre avec pause (3 tests)

`Stopwatch` mesure le temps de jeu ; il se met en pause pendant la lecture d'une fiche.

| Test | Ce qu'il vérifie |
|---|---|
| `pausedTime_isNotCounted` | Le temps passé en pause n'est pas compté. |
| `pauseAndResume_twice_areHarmless` | Mettre deux fois en pause ou relancer deux fois ne fausse pas le temps. |
| `stop_endsAPauseAndFreezesTheTime` | L'arrêt de fin de partie termine une pause en cours et fige le temps ; une pause après l'arrêt est sans effet. |

## game/QuizLevelsTest — niveaux de jeu (6 tests)

Ces tests vérifient la vraie configuration `assets/game/levels.json` contre les vraies données.

| Test | Ce qu'il vérifie |
|---|---|
| `zonesCoverEveryMapPlaceExactlyOnce` | Les zones Nord et Sud couvrent chaque lieu de la carte, une seule fois. |

| Test | Ce qu'il vérifie |
|---|---|
| `specialUnitsAreTheFourDecidedOnes` | Les unités « spéciales », absentes des niveaux « continent », sont exactement Hong Kong, Macao, Taïwan et les îles Paracel. |
| `levelsHaveTheExpectedSizes` | Nord continent 15, Sud continent 16, Sud complet 20, toute la Chine continent 31, complet 35. |
| `northAndSouthAreBalanced` | Les zones Nord et Sud diffèrent d'au plus un lieu. |
| `everyQuestionTargetsAPlaceWithGeometryAndData` | Chaque question porte sur un lieu qui existe dans les données et sur la carte : aucune question sans réponse possible. |
| `placesMissingFromTheMapAreNeverAsked` | Si un lieu manquait sur la carte, il ne serait jamais demandé. |

## map/HitTesterTest — détection du toucher (14 tests)

`HitTester` répond à la question « quel lieu se trouve sous le doigt ? ». Ces tests utilisent des formes géométriques simples (carrés, lignes) pour vérifier chaque cas délicat.

| Test | Ce qu'il vérifie |
|---|---|
| `pointInside_isFound` | Un point à l'intérieur d'une surface la trouve. |
| `pointInHole_isNotInside` | Un point dans un trou de la surface (une enclave) n'est pas dedans. |
| `pointOnSeparateIsland_belongsToItsShape` | Un point sur une île détachée appartient bien à sa province (Hainan, Taïwan). |
| `pointInSea_withoutTolerance_isNothing` | Un point en mer ne touche rien. |
| `overlappingAreas_theSmallestWins` | Quand deux zones se superposent (un site dans un grand parc), le toucher choisit la plus petite, quel que soit l'ordre des données ; sinon le petit site serait inaccessible. |
| `pointNearSmallIsland_isFoundWithinTolerance` | Près d'une petite île, une tolérance permet de la toucher quand même : c'est ainsi que Macao reste sélectionnable. |
| `toleranceChoosesTheClosestShape` | Dans la zone de tolérance, la forme la plus proche l'emporte. |
| `exactHitWinsOverCloserNeighbourEdge` | Un point vraiment dans une forme la choisit, même si le bord d'une voisine est plus proche. |
| `lineIsFoundNearItsTrace_butIsNotASurface` | Un fleuve se touche près de son tracé, mais n'a pas d'« intérieur ». |
| `openLineHasNoClosingSegment` | Une ligne ouverte n'est pas refermée : il n'y a pas de segment fantôme entre son début et sa fin. |
| `pointBeatsTheSurfaceItSitsOn` | Un sommet ou une ville l'emporte sur la surface sur laquelle il est posé. |
| `acceptFiltersOutShapesThatAreNotInPlay` | Les lieux hors du niveau en cours (grisés) sont ignorés. |
| `areaSumsAllRings` | La surface d'un lieu fait la somme de toutes ses parties (îles comprises). |

| Test | Ce qu'il vérifie |
|---|---|
| `boundsCoverAllRings` | Le cadre d'un lieu englobe toutes ses parties. |

## map/MapViewportTest — zoom et déplacement (5 tests)

`MapViewport` convertit les coordonnées de la carte en pixels à l'écran, et inversement.

| Test | Ce qu'il vérifie |
|---|---|
| `fit_centersTheWholeMap` | Au départ, la carte entière est visible et centrée. |
| `toMap_isTheInverseOfToScreen` | Passer de la carte à l'écran puis revenir redonne le même point. |
| `zoom_keepsThePointUnderTheFingersStill` | Pendant un zoom à deux doigts, le point sous les doigts ne bouge pas. |
| `zoom_isClampedBetweenOverviewAndMaximum` | Le zoom est limité entre la vue d'ensemble et un grossissement maximal. |
| `pan_cannotLoseTheMap` | On ne peut pas faire glisser la carte hors de l'écran au point de la perdre. |

## map/RealMapTest — vraie carte et vraies coordonnées GPS (7 tests)

Ces tests projettent de vraies coordonnées GPS sur la carte livrée. Ils vérifient à la fois que la projection écrite en Kotlin est identique à celle du script Python qui a produit la carte, et que la détection du toucher fonctionne sur les vrais tracés.

| Test | Ce qu'il vérifie |
|---|---|
| `citiesFallInTheirProvince` | La place Tian'anmen est dans Beijing, Lhassa dans le Xizang, Taipei à Taïwan, et ainsi de suite pour 11 villes. |
| `smallUnitsAreReachable` | Hong Kong (Kowloon), Macao et l'île Woody (Paracel) sont touchables, avec 2 km de tolérance pour les deux dernières. |
| `neighbouringMunicipalitiesAreDistinct` | Tianjin et le Hebei qui l'entoure sont bien distingués. |
| `seaAndForeignCountriesAreNothing` | La mer Jaune, la mer des Philippines, la Mongolie et New Delhi ne correspondent à aucun lieu. |
| `allShapesLieInsideTheDeclaredBounds` | Les 35 tracés sont tous dans le cadre déclaré de la carte. |
| `labelPointsLieInsideTheirShape` | Le point où l'on écrit le nom de chaque province est bien à l'intérieur, même pour les formes tordues (Gansu, Mongolie-Intérieure). |
| `northIsUp` | Le nord est en haut : Harbin est au-dessus de Haikou. |

## progress/ProgressTest — records et erreurs fréquentes (9 tests)

`Progress` garde, pour chaque mode et chaque niveau, le meilleur résultat, ainsi que le nombre de fois où chaque lieu a été manqué. Elle est sauvegardée sur le téléphone en JSON.

| Test | Ce qu'il vérifie |
|---|---|
| `firstGame_isARecord` | La première partie d'un niveau est un record. |

| Test | Ce qu'il vérifie |
|---|---|
| `moreCorrectAnswers_beatsAFasterTime` | Plus de bonnes réponses bat un meilleur temps : le score compte avant la vitesse. |
| `sameScore_fasterIsARecord_slowerIsNot` | À score égal, seul un temps plus rapide est un record. |
| `worseGame_keepsThePreviousBest` | Une moins bonne partie ne remplace pas le record. |
| `modesAndLevelsAreKeptSeparately` | Les records sont séparés par mode et par niveau. |
| `mistakesReplay_doesNotBecomeTheLevelBest_butCountsMisses` | Une partie « rejouer les erreurs » ne compte pas comme record (elle est plus courte), mais ses erreurs sont comptées. |
| `missCounts_accumulateOncePerGame` | En mode Éliminer, trois erreurs sur le même lieu dans une partie ne comptent qu'une fois ; les lieux les plus manqués sont classés. |
| `completion_sumsBestScoresOverLevels` | Le pourcentage de réussite d'une catégorie additionne les meilleurs scores de tous ses niveaux. |
| `jsonRoundTrip_andTolerantReading` | La progression sauvegardée puis relue est identique ; une sauvegarde abîmée ou absente redonne une progression vierge, sans plantage. |

## ExampleUnitTest — exemple du modèle (1 test)

| Test | Ce qu'il vérifie |
|---|---|
| `addition_isCorrect` | 2 + 2 = 4. Test d'exemple créé automatiquement par Android Studio avec le projet ; il sert seulement à vérifier que la chaîne de tests fonctionne. |

## Ce que les tests ne couvrent pas

- **Les écrans** (Jetpack Compose) : vérifiés à la main, sur l'émulateur et sur le téléphone, avec des captures d'écran.
- **La synthèse vocale** : elle dépend du moteur installé sur le téléphone ; on a vérifié sur l'émulateur, dans le journal système, que la lecture démarre et se termine.
- **La sauvegarde sur le téléphone** (DataStore) : seul le format JSON de la progression est testé.
- **L'exactitude des textes des fiches** : les tests vérifient leur forme (présence, longueur, identifiants), pas la vérité de leur contenu.